Câu hỏi kỹ thuật về AI Assistant mà người nghe có thể hỏi (và cách trả lời)

Bổ sung cho `PW_AI_Assistant_QA.md`. File này tập trung vào câu hỏi **kỹ thuật**: dữ liệu, vector, tìm kiếm, mô hình, bảo mật, vận hành.

0. Nguyên tắc khi trả lời câu hỏi kỹ thuật

1. **Trả lời theo ba tầng:** (a) nó là gì, (b) em đề xuất dùng gì cho PW, (c) trong bản demo em làm gì. Tầng (c) luôn nói thật.
2. **BA không phải kỹ sư triển khai.** Em đề xuất và nêu tiêu chí chọn; nhóm kỹ thuật chốt sau khi thử. Câu an toàn khi không chắc: *"Phần này em đề xuất hướng và tiêu chí, con số cụ thể nhóm kỹ thuật cần đo sau khi thử nghiệm."*
3. **Không nêu con số chưa đo** (độ chính xác, chi phí, mili-giây). Con số thời gian trong tài liệu này chỉ là cỡ thường gặp, không phải kết quả đo của PW.
4. **Bản demo:** tìm tài liệu bằng BM25 (từ khoá) trong trình duyệt, **không có embedding và không có vector database**, bước viết câu trả lời là giả lập. Nếu bị hỏi về bản demo, nói đúng như vậy.

---

1. Chuyển dữ liệu thành vector database

Q1. Nếu em muốn chuyển dữ liệu (tài liệu) thành vector database thì em dùng chương trình gì? Quy trình có năm bước, mỗi bước một công cụ: 1. **Thu thập và làm sạch tài liệu:** tài liệu viết bằng Markdown (hoặc lấy từ Notion, Confluence, Google Docs), bỏ phần thừa. 2. **Chia đoạn (chunking):** cắt theo tiêu đề và mục, thường 300–800 từ vựng (token) mỗi đoạn, chồng lấn nhẹ khoảng 10–15% để không cắt đứt ý. 3. **Tạo vector (embedding):** gọi một **mô hình embedding** qua API, ví dụ Voyage AI, OpenAI (text-embedding), Cohere (loại đa ngôn ngữ), hoặc mô hình mã nguồn mở như BGE-M3 hay multilingual-e5 chạy bằng thư viện sentence-transformers. 4. **Lưu vào cơ sở dữ liệu vector:** em đề xuất **PostgreSQL + pgvector**; phương án khác là Pinecone (dịch vụ trọn gói), Qdrant, Weaviate, Chroma, Milvus. 5. **Chạy tự động:** một script Python (hoặc Node) dùng SDK của nhà cung cấp, hoặc thư viện **LangChain / LlamaIndex** có sẵn bộ đọc tài liệu và bộ chia đoạn; chạy theo lịch hoặc khi có bản mới bằng cron, GitHub Actions hoặc Airflow. Trong sơ đồ D7.1 đây là các bước từ "làm sạch và chia đoạn" đến "lưu vector kèm metadata", rồi **chạy bộ câu hỏi kiểm thử trước khi bật chỉ mục mới**.

Q2. Vector (embedding) là gì? Giải thích đơn giản. Là dãy số biểu diễn **ý nghĩa** của một đoạn chữ. Hai đoạn có ý gần nhau thì vector gần nhau, nên tìm được "câu hỏi về giảm giá năm hai" dù từ ngữ không giống hệt tài liệu.

Q3. Vì sao chọn pgvector mà không phải Pinecone? Quy mô của PW nhỏ (hàng chục đến vài trăm tài liệu), PW cần một cơ sở dữ liệu quan hệ cho phần còn lại, nên dùng chung Postgres **ít hệ thống hơn để vận hành** và dễ kết hợp lọc theo quyền. Pinecone hợp khi cần mở rộng lớn và không muốn tự vận hành. Đây là đề xuất, nhóm kỹ thuật chốt.

Q4. Chia đoạn bao nhiêu là hợp lý? Sai thì sao? Đoạn quá ngắn thì mất ngữ cảnh, quá dài thì lẫn nhiều ý và tốn chi phí. Em đề xuất bắt đầu theo tiêu đề và mục, 300–800 token, rồi **chỉnh bằng bộ câu hỏi kiểm thử** (xem đoạn nào được tìm đúng). Con số chốt dựa trên thử nghiệm.

Q5. Metadata dùng để làm gì? Gắn cho mỗi đoạn: mã tài liệu, phiên bản, **đối tượng được xem (public, member, admin)**, ngôn ngữ, ngày. Dùng để lọc theo quyền ngay lúc tìm, dẫn nguồn trong câu trả lời, và xoá đoạn của bản cũ khi tài liệu đổi.

Q6. Khi tài liệu đổi thì xử lý thế nào? Có phải làm lại hết không? Chỉ nhúng lại các đoạn thuộc tài liệu đã đổi, rồi chạy bộ kiểm thử, đạt thì chuyển sang chỉ mục mới và xoá đoạn cũ; không đạt thì giữ chỉ mục cũ và báo người phụ

trách. **Nếu đổi mô hình embedding thì phải nhúng lại toàn bộ**, vì vector của hai mô hình không so sánh được với nhau.

Q7. Dữ liệu động như điểm, hạng, NFT có đưa vào vector database không? Không. Đó là dữ liệu có cấu trúc, đổi từng phút và cần đúng tuyệt đối; tìm theo "gần nghĩa" dễ sai số hoặc sai tên, và phải lập chỉ mục lại mỗi giao dịch. Chúng đọc qua **hàm truy vấn chỉ đọc** trên cơ sở dữ liệu của PW.

Q8. Dữ liệu trên blockchain thì AI lấy thế nào? Qua cơ sở dữ liệu của PW (đã đồng bộ), hoặc hỏi trực tiếp blockchain thông qua nhà cung cấp node như Alchemy (có API tra NFT và chủ sở hữu). Cả hai đều là hàm đọc, không phải nội dung đưa vào kho tài liệu.

Q9. Có thể dùng luôn công cụ "Claude Projects" hay "Custom GPT" thay vì tự dựng không? Dùng được để **thử ý tưởng nhanh**. Không dùng cho sản phẩm vì không kiểm soát được quyền dữ liệu theo từng người dùng, nhật ký, đo chất lượng, và việc nối với dữ liệu sống của PW.

Q10. Tài liệu hiện có 19 mục, tại sao còn cần vector database? Thật ra với cỡ này có thể **đưa toàn bộ tài liệu vào lời nhắc hệ thống** (kèm bộ nhớ đệm lời nhắc), chưa cần vector. Em thiết kế sẵn luồng D7.1 vì khi kho tài liệu lớn lên (BRD, SRS, chính sách theo thẩm quyền, nhiều ngôn ngữ) cách đó không đủ. Lộ trình: giai đoạn 1 dùng lời nhắc, giai đoạn 2 chuyển sang tìm lai.

---

2. Tìm kiếm và xếp hạng kết quả

Q11. Tìm kiếm trong demo hoạt động thế nào? BM25: thuật toán xếp hạng theo **từ khoá** (từ nào hiếm và xuất hiện nhiều trong đoạn thì điểm cao). Chạy trong trình duyệt, có bảng từ đồng nghĩa nhỏ và ngưỡng điểm, và chỉ trả lời khi độ phủ từ khoá đủ. Bản thật dùng embedding.

Q12. Tìm lai (hybrid search) là gì và vì sao cần? Kết hợp **vector** (hiểu nghĩa) với **từ khoá** (khớp chính xác). Mã như `#PW-2026-INV-10233` hay `PW-ORG-0081` và tên pháp nhân là chuỗi chính xác, embedding thuần thường trượt. Hai danh sách kết quả được gộp bằng kỹ thuật như **Reciprocal Rank Fusion**.

Q13. Độ giống giữa hai vector đo bằng gì? Thường là **độ tương đồng cosine** (góc giữa hai vector). Cơ sở dữ liệu vector dùng chỉ mục xấp xỉ (như **HNSW**, pgvector có hỗ trợ) để tìm nhanh trong nhiều vector mà không phải so từng cái.

Q14. Lấy bao nhiêu đoạn đưa vào mô hình (top-k)? Em đề xuất lấy khoảng 10 đoạn rồi **xếp hạng lại (rerank)** còn 3–5 đoạn tốt nhất. Ít quá thì thiếu ý, nhiều quá thì nhiễu và tốn chi phí. Con số chốt qua thử nghiệm.

Q15. Reranker là gì? Một mô hình thứ hai chấm lại mức liên quan giữa câu hỏi và từng đoạn tìm được, chính xác hơn bước tìm nhanh. Ví dụ Cohere Rerank, Voyage rerank, hoặc mô hình mã nguồn mở bge-reranker.

Q16. Làm sao bảo đảm người dùng thường không đọc được tài liệu dành cho admin? Lọc theo quyền ngay lúc tìm bằng metadata (public, member, admin). Lọc sau khi mô hình đã nhận đoạn là quá muộn.

Q17. Nếu không tìm được đoạn nào đủ liên quan? Nếu đã có số liệu sống thì vẫn trả lời bằng số liệu đó. Nếu cả hai nguồn đều trống thì trả lời rõ là không có thông tin, **không đoán**.

---

3. Mô hình ngôn ngữ và lời nhắc (prompt)

Q18. Em chọn mô hình nào và dựa trên tiêu chí gì? Đề xuất Claude cho phần viết câu trả lời (mô hình cỡ vừa) và một mô hình cỡ nhỏ cho phân loại câu hỏi. Tiêu chí: bám sát tài liệu, tiếng Việt, độ trễ, chính sách dữ liệu, giá. Phương án khác GPT, Gemini. Nhóm kỹ thuật so sánh bằng chính bộ câu hỏi kiểm thử của PW.

Q19. Lời nhắc gồm những gì? Bốn phần: (1) quy tắc của trợ lý (chỉ dựa vào dữ liệu được đưa vào, không tư vấn đầu tư, dẫn nguồn); (2) số liệu sống của người hỏi và của nền tảng; (3) các đoạn tài liệu kèm mã nguồn; (4) câu hỏi. Demo có nút "How this was answered" cho xem đúng lời nhắc này.

Q20. Tool use / function calling là gì? Cho mô hình biết có những **hàm** nào (ví dụ lấy điểm, lấy danh sách NFT của tôi). Khi cần, mô hình yêu cầu gọi hàm, **máy chủ** chạy hàm theo quyền người dùng và trả kết quả lại cho mô hình viết câu trả lời. Mô hình không tự truy cập cơ sở dữ liệu.

Q21. Tại sao không đưa cả cơ sở dữ liệu cho mô hình, hoặc để mô hình tự viết câu SQL? Rủi ro lộ dữ liệu người khác và câu lệnh sai hoặc nguy hiểm, và tốn chi phí khi dữ liệu lớn. Dùng các hàm chỉ đọc đã định sẵn, có kiểm quyền, an toàn hơn.

Q22. Cửa sổ ngữ cảnh (context window) là gì và vì sao không nhét hết tài liệu? Là lượng chữ tối đa mô hình nhận trong một lần. Nhét hết thì tốn chi phí và chậm, mô hình dễ bỏ sót ý ở giữa. Với 19 tài liệu hiện nay thì nhét được; khi kho lớn thì phải tìm và chỉ đưa đoạn liên quan.

Q23. Bộ nhớ đệm lời nhắc (prompt caching) để làm gì? Phần lời nhắc cố định (quy tắc, tài liệu) được lưu lại để lần gọi sau rẻ và nhanh hơn. Hợp với giai đoạn 1 khi đưa toàn bộ tài liệu vào lời nhắc.

Q24. Fine-tune khác RAG thế nào? Fine-tune thay đổi bản thân mô hình, tốn công và **khó cập nhật khi quy tắc đổi**, không có nguồn để kiểm. RAG đổi tài liệu là cập nhật ngay và có dẫn nguồn. PW có quy tắc đổi thường xuyên nên dùng RAG.

Q25. Làm sao đo "không bịa" (hallucination)? Đo hai thứ: câu trả lời có **bám vào nguồn được đưa vào** không (groundedness/faithfulness), và có đúng số liệu không. Em dùng bộ câu hỏi kiểm thử có đáp án mong đợi, kể cả loại câu "phải từ chối" và "phải nói không có dữ liệu". Công cụ đo như Ragas, Promptfoo, TruLens khi có bản thật.

Q26. Ngôn ngữ: người dùng hỏi tiếng Việt thì sao? Demo chỉ hiểu tiếng Anh. Bản thật cần mô hình ngôn ngữ và mô hình embedding **đa ngôn ngữ**, và tài liệu nên có bản tiếng Việt hoặc chấp nhận tìm xuyên ngôn ngữ. Cần chốt sớm vì đổi mô hình embedding phải nhúng lại toàn bộ.

---

4. Tích hợp và bảo mật

Q27. Khoá API để ở đâu? Chỉ ở **máy chủ trung gian** (hàm serverless như Vercel, Cloudflare Workers, AWS Lambda). Không bao giờ ở trình duyệt. Trình duyệt chỉ gọi một điểm cuối của PW với câu hỏi.

Q28. Máy chủ biết ai đang hỏi bằng cách nào? Từ **phiên đăng nhập** trên máy chủ (mã người dùng và vai trò). Không tin mã người dùng do trình duyệt gửi lên, vì có thể bị giả mạo.

Q29. Prompt injection là gì và PW phòng thế nào? Là việc người dùng (hoặc nội dung tài liệu) chèn câu lệnh để mô hình làm sai ("bỏ qua mọi quy tắc..."). Phòng: coi mọi chữ người dùng nhập và nội dung tài liệu là **dữ liệu, không phải lệnh**; kiểm tra đầu vào bằng quy tắc và một mô hình nhỏ; kiểm tra câu trả lời (có nguồn, không lộ dữ liệu người khác); mô hình **không được quyền ghi dữ liệu**. Bản demo chỉ chặn bằng quy tắc nên dễ bị lách; em nói rõ điều này.

Q30. Dữ liệu cá nhân gửi cho nhà cung cấp mô hình có an toàn không? Chỉ gửi phần cần thiết (số liệu của chính người hỏi), che thông tin nhạy cảm trong nhật ký, chọn nhà cung cấp có cam kết **không dùng dữ liệu để huấn luyện** và lưu trữ tối thiểu, và để Compliance rà soát hợp đồng. KYC (giấy tờ) **không bao giờ** đưa vào trợ lý.

Q31. AI có thể thực hiện giao dịch không? Không. Theo BRD trợ lý chỉ đọc. Giao dịch luôn do người dùng tự xác nhận. (Prototype có phần điền sẵn đơn mua, người dùng vẫn tự bấm thanh toán; phần này ngoài BRD.)

Q32. Chống lạm dụng thế nào (spam, tốn chi phí)? Giới hạn số câu hỏi mỗi người, giới hạn độ dài câu hỏi, yêu cầu đăng nhập cho tính năng dùng dữ liệu cá nhân, và theo dõi chi phí theo người dùng.

Q33. Prompt injection qua tài liệu thì sao (tài liệu bị chèn lệnh)? Tài liệu đi qua bước **duyệt** trước khi nạp (D7.1), lời nhắc dặn mô hình chỉ coi đoạn tài liệu là dữ liệu tham khảo, và đoạn chỉ có đối tượng xem đúng quyền mới được tìm thấy.

Q34. Còn ghi nhật ký thì sao? Có vi phạm quyền riêng tư không? Ghi câu hỏi, đoạn tìm được, câu trả lời, độ trễ để rà soát và cải thiện, **che dữ liệu nhạy cảm**, có thời hạn lưu và chỉ vai trò được phép mới xem được. Công cụ như Langfuse hoặc LangSmith.

---

5. Vận hành, hiệu năng, đo lường

Q35. Thời gian phản hồi 2–5 giây phân bổ ra sao? Cỡ thường gặp: embedding câu hỏi và tìm kiếm vài chục đến vài trăm mili-giây, xếp hạng lại thêm một chút, **phần lớn thời gian là mô hình ngôn ngữ**. Cách giữ trong 2–5 giây: truyền dần câu trả lời (người dùng thấy chữ đầu tiên rất sớm), dùng mô hình vừa đủ, ít đoạn nhưng đúng, bộ nhớ đệm lời nhắc. Đây là yêu cầu em cần **đo thật** khi có bản chạy mô hình.

Q36. Nếu nhà cung cấp mô hình bị lỗi hoặc quá tải? Hạ cấp có kiểm soát: hiện thông báo "trợ lý tạm thời không phản hồi", kèm đường dẫn tới trang hướng dẫn và liên hệ hỗ trợ; thử lại có giới hạn; không làm ảnh hưởng các chức năng chính (mua, xác minh). Yêu cầu khả dụng nền tảng là 99,5% mỗi tháng (NFR-007).

Q37. Chịu tải bao nhiêu người cùng lúc? Yêu cầu của BRD: 200 người dùng đồng thời trong năm đầu (NFR-006). Hàm serverless co giãn theo số yêu cầu; điểm nghẽn cần kiểm là hạn mức gọi của nhà cung cấp mô hình.

Q38. Đo chất lượng liên tục bằng cách nào? Bộ câu hỏi kiểm thử **chạy mỗi khi đổi tài liệu, lời nhắc hoặc mô hình**; mỗi lỗi gặp thêm một câu vào bộ. Chỉ số: tỷ lệ tìm đúng đoạn (recall@k), bám nguồn, từ chối đúng, độ trễ. Câu quan trọng có người chấm. Demo hiện có 33 câu, đạt hết.

Q39. Ai sở hữu nội dung tài liệu và ai duyệt? Một người phụ trách nội dung (đề xuất Product Owner phối hợp Compliance). Tài liệu viết trong kho Git, duyệt bằng pull request, nạp tự động theo D7.1, kiểm thử trước khi bật.

Q40. Phiên bản chỉ mục là gì và vì sao cần? Mỗi lần nạp tạo một bản chỉ mục mới; chỉ chuyển sang bản mới khi bộ kiểm thử đạt, nên không bao giờ trả lời bằng chỉ mục hỏng hoặc lai giữa hai phiên bản.

---

6. Lựa chọn kiến trúc (câu hỏi "tại sao không...")

Q41. Tại sao không dùng mô hình mã nguồn mở chạy trên máy của PW? Có thể, nếu yêu cầu dữ liệu không được rời hệ thống. Đổi lại phải tự vận hành máy chủ có GPU, theo dõi và cập nhật mô hình, và chất lượng tiếng Việt cần kiểm. Em đề xuất bắt đầu bằng dịch vụ API, để mở đường chuyển sau nếu Compliance yêu cầu.

Q42. Tại sao không dùng LangChain cho toàn bộ? Quy trình chỉ vài bước (kiểm tra, tìm, ghép lời nhắc, gọi mô hình, kiểm tra), gọi trực tiếp SDK ít mã và ít phụ thuộc hơn. LangChain hoặc LlamaIndex hữu ích cho **phần nạp tài liệu** (bộ đọc, bộ chia đoạn) hơn là cho phần trả lời.

Q43. Có cần "agent" (AI tự lập kế hoạch nhiều bước) không? Chưa. Trợ lý chỉ cần quy trình cố định: kiểm tra, gọi hàm đọc, tìm tài liệu, trả lời. Agent khó kiểm soát và khó đo; chỉ cân nhắc khi có nhu cầu nhiều bước thật sự.

Q44. Tại sao chia hai sơ đồ D7.1 và D7.2? Hai việc khác nhau về thời điểm và người chịu trách nhiệm: nạp tri thức (khi tài liệu đổi, do người phụ trách nội dung) và trả lời (mỗi câu hỏi, tự động). Tách ra để kiểm thử và phân quyền rõ ràng.

Q45. Khi cần mở rộng thêm nhiều loại dữ liệu (PDF, bảng) thì sao? Thêm bước đọc và làm sạch cho từng loại ở D7.1; phần trả lời không đổi. PDF dạng ảnh cần nhận dạng chữ (OCR) trước. Bảng và số liệu có cấu trúc nên đi qua hàm truy vấn thay vì đưa vào vector.

Q46. Em đã thử nghiệm những gì, chưa làm những gì? Đã làm: demo chạy trong trình duyệt với tìm kiếm BM25, quy tắc chặn, hàm đọc số liệu sống, bộ 33 câu kiểm thử, bảng "How this was answered". Chưa làm: embedding và vector database thật, gọi mô hình ngôn ngữ thật, máy chủ trung gian, đo độ trễ và chi phí. Em ghi rõ ranh giới này trong tài liệu thiết kế.

---

7. Bảng thuật ngữ nhanh (đọc lại trước khi trình bày)

Thuật ngữ	Một câu giải thích
RAG	Tìm đoạn tài liệu liên quan rồi đưa vào lời nhắc để mô hình trả lời dựa trên đó
Chunk	Một đoạn tài liệu sau khi chia nhỏ
Embedding	Dãy số biểu diễn ý nghĩa của một đoạn chữ
Vector database	Cơ sở dữ liệu lưu vector và tìm các vector gần nhau
Cosine similarity	Cách đo hai vector gần nhau (góc giữa chúng)
HNSW	Kiểu chỉ mục tìm gần đúng nhanh trong nhiều vector
BM25	Thuật toán xếp hạng theo từ khoá
Hybrid search	Kết hợp vector và từ khoá
Rerank	Chấm lại mức liên quan của các đoạn đã tìm được
Top-k	Số đoạn tốt nhất được lấy
Token	Đơn vị chữ mô hình đếm (khoảng một từ ngắn hoặc một phần từ)
Context window	Lượng chữ tối đa mô hình nhận trong một lần
Prompt	Toàn bộ chữ gửi cho mô hình (quy tắc, dữ liệu, câu hỏi)
Function calling / Tool use	Mô hình yêu cầu chạy một hàm, máy chủ chạy rồi trả kết quả
Hallucination	Mô hình bịa thông tin không có trong nguồn
Grounding	Buộc câu trả lời dựa trên nguồn được đưa vào
Prompt injection	Chèn lệnh để làm mô hình làm sai quy tắc
Prompt caching	Lưu phần lời nhắc cố định để lần sau rẻ và nhanh hơn
Streaming	Gửi câu trả lời từng phần để người dùng thấy chữ ngay
Fine-tuning	Huấn luyện thêm bản thân mô hình, khó cập nhật khi quy tắc đổi
Guardrails	Lớp kiểm tra chặn câu hỏi hoặc câu trả lời không phù hợp